# Cloud Workload Execution Models

## A Comparative Analysis of Container and FaaS Scheduling in a Cloud Environment

Project Report

Gianvito Alcamo
Valerio Triolo
Gabriele Giudici

University of Pisa
MSc. in Artificial Intelligence and Data Engineering
Performance Evaluation of Computer Systems and Networks Course
Academic Year 2025–2026






Scuola
di Ingegneria

# Contents

**Contents** 1

**GitHub Repository** 2

**1 Introduction** 3
- 1.1 Problem description . . . 3
- 1.2 Objectives . . . 3
- 1.3 Performance indexes . . . 3

**2 Modeling** 4
- 2.1 Assumptions . . . 4
- 2.2 Preliminary validation . . . 4
- 2.3 Factors . . . 4
- 2.4 Theoretical model . . . 5

**3 Implementation** 5
- 3.1 Aggregated User Source . . . 6
- 3.2 Virtual Server and Fluid Capacity . . . 6

**4 Verification** 6
- 4.1 Degeneracy Test . . . 6
  - 4.1.1 Comments on Results . . . 7
- 4.2 Consistency Test . . . 7
  - 4.2.1 Test Configuration . . . 7
  - 4.2.2 Results Analysis . . . 8
  - 4.2.3 Visual Validation . . . 8
- 4.3 Continuity Test . . . 9
  - 4.3.1 Test Configuration . . . 9
  - 4.3.2 Results Analysis . . . 10
  - 4.3.3 Visual Validation . . . 10
- 4.4 Validation through theoretical model . . . 11
  - 4.4.1 Test Configuration . . . 11
  - 4.4.2 Analytical Reference . . . 11
  - 4.4.3 Results Analysis . . . 11
  - 4.4.4 Discussion . . . 12
- 4.5 Valgrind Test . . . 12

**5 Calibration** 12
- 5.1 Cloud Computing Infrastructure Dimensioning . . . 12
  - 5.1.1 Server Specifications . . . 13
  - 5.1.2 Physical-to-Virtual Resource Mapping . . . 13
- 5.2 Factors Dimensioning . . . 13
  - 5.2.1 Required Capacity ($R$) . . . 13
  - 5.2.2 Execution Duration ($D$) . . . 13
  - 5.2.3 Dimensioning of Inter-Arrival Time ($T$) . . . 14
  - 5.2.4 Analytical Derivation of the User Range ($N$) . . . 14
  - 5.2.5 Derivation of the User Range ($N$) . . . 14
  - 5.2.6 Dimensioning of Function Granularity ($K$) . . . 15
  - 5.2.7 Dimensioning of Instantiation and Termination Overheads ($I_{cont}$ and $I_{FaaS}$) 15
- 5.3 Scaled Model for Efficient Simulation . . . 16
  - 5.3.1 Scaling Methodology . . . 16
  - 5.3.2 Scaled Parameters . . . 16
  - 5.3.3 Refinement of the Simulation Range . . . 16
- 5.4 Warm-up time . . . 17

**6 Simulation** **18**

**7 Conclusions** **19**

**References** **21**

## GitHub Repository

The GitHub repository containing all the project files is located at:

https://github.com/ga83b/Performance_Evaluation_CSN_private

*The repository is currently private, to access the repository contact the authors of this documentation.*

# 1 Introduction

## 1.1 Problem description

A cloud computing infrastructure provides a total computing capacity of $C$ GHz and receives requests for workload execution from $N$ users. Each user generates a new workload every $T$ seconds, and each workload requires a computing capacity of $R$ $GHz$ for $D$ seconds. $T$, $R$ and $D$ are IID random variables whose distribution is specified below. Each user submits its workloads to the cloud infrastructure according to one of the following execution models:

- Container-based model: each workload is executed within a long-term container, which runs at $R$ GHz for $D$ seconds. The container takes $I_{cont}$ seconds to be instantiated and $I_{cont}$ seconds to be terminated.
- Function as a Service (FaaS) model: each workload is divided into $K$ functions of equal duration, and functions are executed sequentially, i.e., function $k$ is started only after function $k-1$ has completed. Each function is executed within a short-term container that runs at $R$ GHz for the time needed to execute that function only. The container takes $I_{FaaS}$ seconds to be instantiated and $I_{FaaS}$ seconds to be destroyed. Moreover, $I_{FaaS} < I_{cont}$.

For each incoming workload or function, the cloud infrastructure checks if there is enough capacity to execute it immediately. If capacity is insufficient, the workload or function is placed into a First Come First Served (FCFS) queue and starts execution as soon as enough capacity becomes available. A preliminary scheme of the system is presented in Figure 1.

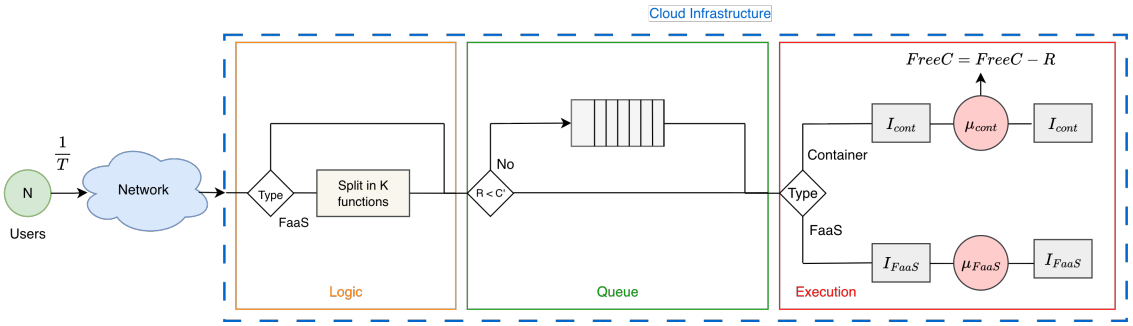



Figure 1: Scheme of the system

## 1.2 Objectives

- Evaluating the total execution time of workloads for various values of $N$ and $K$.
- Comparing the performance between container-based model and Function as a Service model.

## 1.3 Performance indexes

To evaluate the performance of the system we considered these two metrics both in container-based model and in FaaS model:

- The total execution time, in particular the mean response time $E[R]$, the difference between the arrival time of the workload and its completion time
- The utilization $\rho$ of the system, defined as the percentage of time for which the system is busy

# 2 Modeling

## 2.1 Assumptions

- Capacity $C$ is fixed and shared among all types of workloads
- There is no workload loss due to a noisy channel
- Smaller workloads cannot "jump ahead" bigger workloads in the queue
- Thus, cloud infrastructure queue is modeled as a strict infinite FCFS queue, with no dropping.
- The timings $I_{cont}$ and $I_{FaaS}$ are pure overheads that do not consume any capacity $R$.
- The type of a workload is decided uniformly random basing on $p$, the probability of generating a workload of FaaS, intended as the portion of requests that are generated as FaaS-based workloads.
- The total execution time is evaluated as $T_{total} = T_{wait} + T_{exec} + T_{overheads}$ and $T_{overheads}$ doesn't depend on $N$ and $K$. Indeed, the total time a workload (or a single FaaS function) occupies the allocated computing capacity $R$ is defined just by the actual computation duration $D$. Thus, the service time, for a full workload container-based is $\mu_{cont} + 2I_{cont}$ and for a workload FaaS based is $\mu_{FaaS} + 2I_{FaaS}$

## 2.2 Preliminary validation

- The unlimited capacity of the queue is reasonable, as the system provides no way to handle dropped workloads. Furthermore, this analysis focuses on the latency of the cloud infrastructure across different models (container and FaaS), not on the probability of dropping workloads.
- The assumption that workloads cannot preempt others during execution is reasonable, as it simplifies resource management respecting FCFS policy.

## 2.3 Factors

- $N$, number of users the system can serve
- $K$, number of functions constituting a workload, also called the function granularity.
- $C$, the total capacity.
- $R$ the capacity taken by a workload.
- $p$, the probability that a workload will be executed using the FaaS model.
- $I_{cont}$, the overhead time to create a container to execute a workload.
- $I_{FaaS}$, the overhead time to create a short-term container to execute a function.
- $\frac{1}{T}$, the workload arrival rate, as $T$ is the mean of the inter-arrival time distribution,
- $\frac{1}{D}$, the workload service rate, as $D$ is the mean of the service time distribution

### 2.4 Theoretical model

The goal is to compare the performance between container-based model and Function as a Service mode by varying the different values of $N$ and $K$. The arrivals from each of the $N$ independent users are modeled as a Poisson process so that

$$\lambda_{user} = \frac{1}{T}$$

thus the aggregate arrival rate is

$$\Lambda_{agg} = \frac{N}{T} = N \cdot \lambda_{user}$$

Firstly, it was decided to approximate the complex cloud workload system as a simpler **M/M/c** queueing model to allow for analytical tractability and comparison, despite the presence of non-markovian characteristics like the uniform distribution for capacity requirement $R$ and the fixed overhead times $I_{cont}$ and $I_{FaaS}$, which will lead to model as an approximation of a $M/G/c$ queue. Indeed, the service time, in this scenario, is just $D$ and not $D + 2I_{cont}$ or $D + 2KI_{FaaS}$. We can compare the $M/M/c$ setting with simulation assuming that $I_{FaaS}$ and $I_{cont}$ are equal to 0, negletting the overheads. Secondly, the system's ability to process multiple workloads concurrently, constrained by the total capacity $C$, is best represented by the $c$ servers in the $M/M/c$ model, where $c$ is defined as the maximum number of workloads that can execute in parallel approximated as $\frac{C}{R_{min}}$. All $c$ servers are considered identical with rate $\mu = \frac{1}{D}$ as shown in Figure 2.

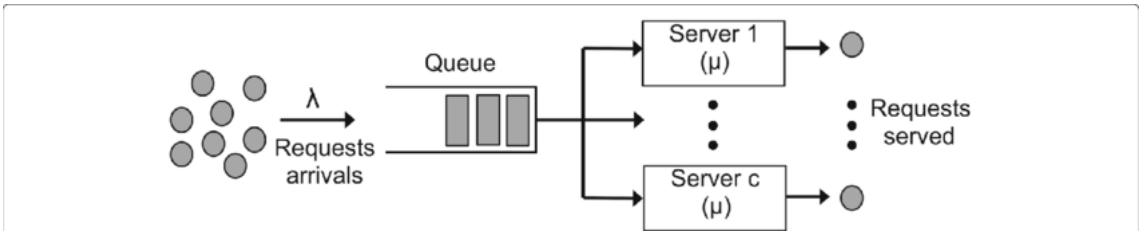



Figure 2: Representation of a M/M/c system. Figure taken from: Research gate

Then we can calculate these metrics and compare them to the results coming from the simulation:

- The waiting time $E[W]$, using the Little's Law, is defined as:

$$E[W] = \frac{E[N_w]}{\lambda_{users}} = \frac{u^c}{c!} \cdot p_0 \cdot \frac{1/(c \cdot \mu)}{(1-\rho)^2}$$

  where $u = \frac{\lambda}{\mu}$ and

$$p_0 = \frac{1}{\sum_{k=0}^{c-1} \frac{u^k}{k!} + \frac{u^c}{c!} \cdot \frac{1}{1-\rho}}$$

- The response time E[R] is defined as:

$$E[R] = E[W] + \frac{1}{\mu}$$

- Utilization $\rho$, defined as $\rho = \frac{\lambda_{users}}{c\mu}$

## 3 Implementation

This section details the architectural choices adopted to describe the system that we implemented for our simulations using OMNeT++. The system implementation follows the logical scheme presented in Figure 3.

We focused on realizing the system as simple as possible, and we also tried to represent the virtualization layer typical of Cloud environments.

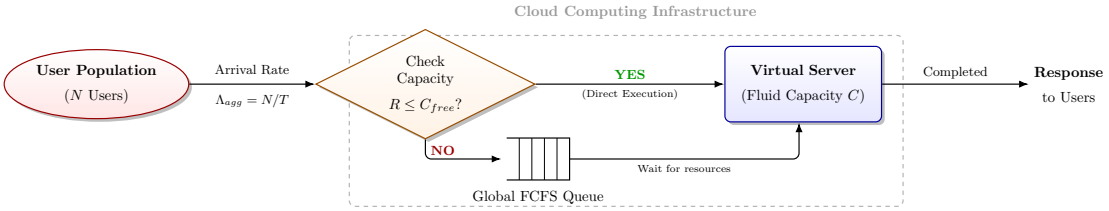



Figure 3: Implemented System Scheme.

### 3.1 Aggregated User Source

Instead of instantiating $N$ distinct client modules—which would introduce unnecessary complications (as far as our analysis is concerned)—we modeled the user population as a single aggregated generator. Given that each of the $N$ independent users generates workloads according to a renewal process with rate $\lambda_{user} = 1/T$, the superposition of these independent processes is modeled as a single global arrival stream. The resulting **Aggregate Arrival Rate** is defined as:

$$\Lambda_{agg} = \sum_{i=1}^{N} \frac{1}{T} = \frac{N}{T} \tag{1}$$

This approach reduces the system complexity while maintaining statistical equivalence.

### 3.2 Virtual Server and Fluid Capacity

The underlying physical cluster is abstracted into a **Single Virtual Server** with **Fluid Capacity** $C$. This design choice is not merely a simplification but an architectural adaptation to the Cloud Computing paradigm, where virtualization layers (e.g., Hypervisors, Kubernetes) mask the physical fragmentation of hardware.

Users interact with a unified pool of resources rather than specific physical machines. Consequently, the simulator manages capacity as a continuous variable:

$$C_{free}(t) = C_{total} - \sum_{i \in \text{Running}} R_i \tag{2}$$

When a workload starts, it consumes a "chunk" of capacity equal to $R$; when it finishes, that capacity is released back to the pool.

## 4 Verification

### 4.1 Degeneracy Test

The Degeneracy Test evaluates the simulator's robustness and logical correctness under extreme or boundary conditions. The goal is to verify that the system reacts predictably to limit-case inputs, avoiding unexpected crashes and producing outputs consistent with the physics of the system.

We conducted a qualitative analysis by varying specific configuration parameters to their logical extremes. The observations are summarized in Table 1.

| Parameter | Value | Expected Behavior | Observed Behavior |
|---|---|---|---|
| **Users ($N$)** | 0 | System idle, no jobs generated. | Zero statistics collected. |
| | 1 | No inter-user contention. | Zero Waiting Time, minimal Response Time. |
| | 100 | System Saturation ($\lambda \gg \mu$). | Unbounded queue growth, full utilization. |
| **Capacity ($C$)** | 0 | No processing capability. | Queue grows indefinitely, throughput is zero. |
| | 1000 | Infinite Server approximation ($M/G/\infty$). | Zero Waiting Time. No queue. |
| **Interarrival ($T$)** | 0 | Infinite arrival rate. | Simulation doesn't terminate as simulation time is not advancing. |
| **Request ($R$)** | 1000 ($R > C$) | Job cannot be served. | Utilization is zero, Queue grows indefinitely. |
| **Parallelism ($P$)** | 0.0 | Only Container mode. | System processes only Container-based requests. |
| | 1.0 | Only FaaS mode. | System processes only FaaS-based requests. |

Table 1: Degeneracy Test Results. Comparison between expected behavior and observed simulation outcome.

#### 4.1.1 Comments on Results

The simulator successfully passed all degeneracy checks, demonstrating robustness and logical consistency. Across all tested boundary conditions (ranging from idle states to extreme saturation and theoretical resource limits) the system behaved according to physical expectations, avoiding runtime errors and producing consistent outputs.

### 4.2 Consistency Test

The goal of the Consistency Test is to verify that the simulator behaves coherently when the input parameters are scaled in a way that should produce predictable changes in the output measures. Specifically, we adopted a *Time Scaling* approach. If we scale the time-related parameters (inter-arrival times and service times) by a factor $k$, the resulting time-based performance metrics (e.g., response time) should scale by the same factor $k$, while other metrics (e.g., utilization, average queue length) should remain invariant, provided that the offered load $\rho$ is kept constant.

#### 4.2.1 Test Configuration

We defined two configurations, setting the overheads ($I_{cont}, I_{FaaS}$) to zero to ensure a pure theoretical comparison:

- **Configuration A:** The baseline scenario with $E[T] = 5s$ and $E[D] = 2.5s$.
- **Configuration B:** The scaled scenario where time parameters are doubled ($E[T] = 10s, E[D] = 5s$).

Table 2 details the specific parameter values used for the simulation runs.

| Parameter | Config A (Test-1) | Config B (Test-2) |
|---|---|---|
| Users ($N$) | 20 | 20 |
| Capacity ($C$) | 100 GHz | 100 GHz |
| **Interarrival Time ($E[T]$)** | **5 s** | **10 s** |
| Request Size ($E[R]$) | 5 GHz | 5 GHz |
| **Duration ($E[D]$)** | **2.5 s** | **5 s** |
| Overheads ($I_{cont}, I_{FaaS}$) | 0 s | 0 s |
| Tasks per Job ($K$) | 4 | 4 |
| Parallelism ($P$) | 0.5 | 0.5 |

Table 2: Parameters for Consistency Test Configurations.

Since both the arrival rate $\lambda \approx \frac{N}{T}$ and the service rate $\mu \approx \frac{1}{D}$ in Configuration B are half the values they have in Configuration A, the system load $\rho = \frac{\lambda}{\mu}$ remains constant across both experiments.

#### 4.2.2 Results Analysis

The simulation was executed with $R = 50$ independent runs. The results are summarized in Table 3.

| Metric | Mean (Config A) | Mean (Config B) | Ratio (B/A) | Verdict |
|---|---|---|---|---|
| Response Time (s) | 2.5016 | 5.0021 | 1.9996 | **PASSED** |
| Waiting Time (s) | 0.0022 | 0.0044 | 1.9833 | **PASSED** |
| Server Utilization | 0.4996 | 0.4995 | 0.9997 | **PASSED** |
| Avg. Queue Length | 0.0089 | 0.0088 | 0.9955 | **PASSED** |

Table 3: Consistency Test Results Summary

The empirical results perfectly align with the theoretical expectations:

1. **Time Scaling:** The *Mean Response Time* $E[R]$ exhibits a ratio of 1.9996, virtually identical to the scaling factor $k = 2$. This confirms the simulator correctly handles event scheduling and duration.
2. **Load Invariance:** The *Server Utilization* $\rho$ and *Queue Length* show a ratio of $\approx 1.0$, confirming that the traffic intensity was successfully preserved.

#### 4.2.3 Visual Validation

Figure 4 provides a macroscopic view of the results. The linear scaling of the response time is clearly visible (orange bars are twice the height of blue bars).

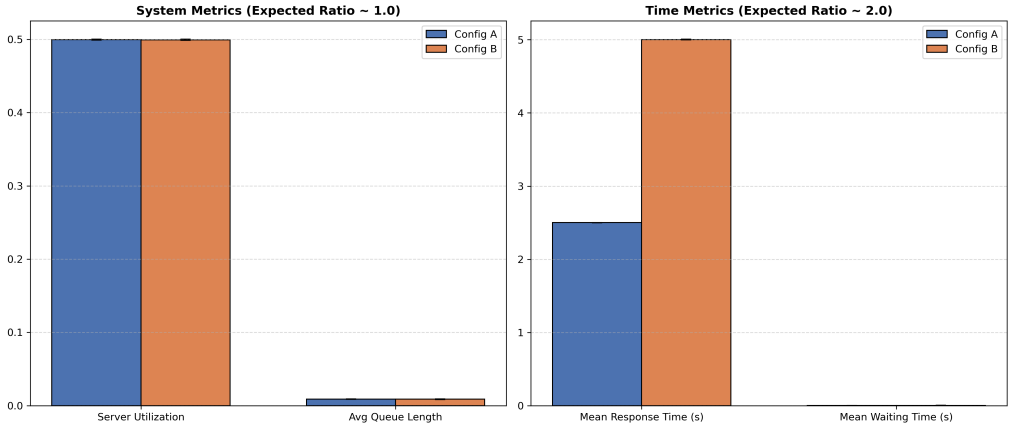



Figure 4: Consistency Test Overview. Comparison of metrics between Config. A (Blue) and Config. B (Orange).

Due to the high precision of the simulator (relative error $< 0.2\%$), the 95% confidence intervals are too narrow to be appreciated in Figure 4 scale. To demonstrate the statistical robustness of the results, Figure 5 presents a "zoomed-in" view focused on the confidence intervals. This visualization confirms that the variance is correctly estimated and the confidence intervals are well-formed around the sample means.

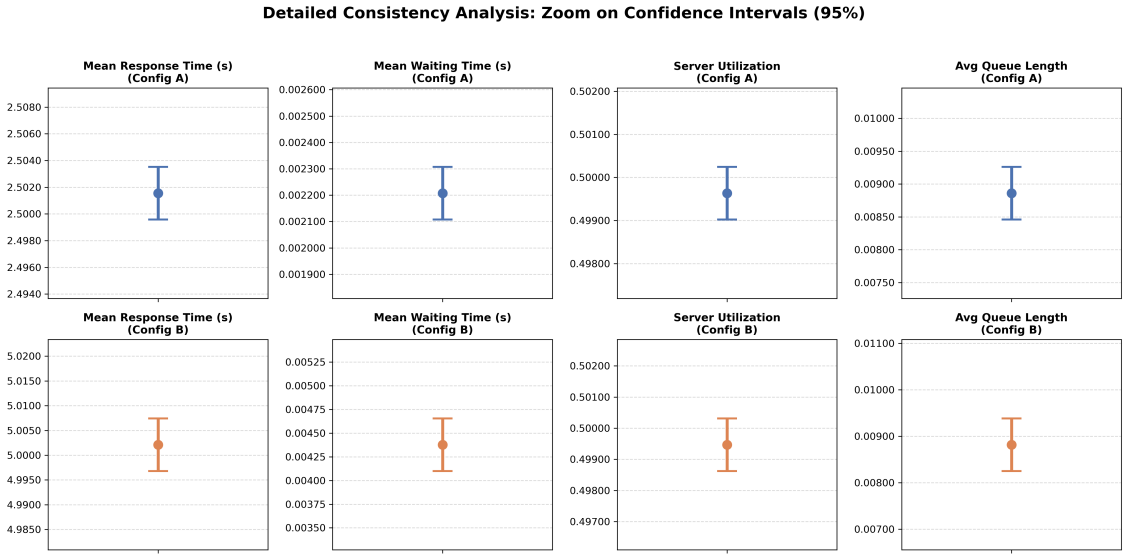



Figure 5: Detailed view of Confidence Intervals (Zoomed Scale). The Y-axis is restricted to a narrow range around the mean to visualize the statistical margins.

## 4.3 Continuity Test

The goal of the Continuity Test is to verify that small variations in input parameters result in small, predictable variations in output performance indices. Furthermore, we verify the property of *Monotonicity*: an increase in the offered load should result in non-decreasing response times and queue lengths.

### 4.3.1 Test Configuration

To evaluate the system under realistic operating conditions, we enabled the overhead models ($I_{cont} = 0.8s, I_{FaaS} = 0.125s$) and configured a scenario with a medium-high system load ($\rho \approx$

72%). We compared two configurations differing by a single user:

- **Configuration A:** Reference scenario with $N = 64$ users.
- **Configuration B:** Incremented scenario with $N = 65$ users.

The specific parameters used for the test are detailed in Table 4.

| **Parameter** | **Config A (Ref)** | **Config B (Inc)** |
|---|---|---|
| Users ($N$) | 64 | 65 |
| Capacity ($C$) | 40 GHz | 40 GHz |
| Interarrival Time ($E[T]$) | 10 s | 10 s |
| Request Size ($E[R]$) | 1 GHz | 1 GHz |
| Duration ($E[D]$) | 3 s | 3 s |
| Overheads ($I_{cont}, I_{FaaS}$) | 0.8 s, 0.125 s | 0.8 s, 0.125 s |
| Tasks per Job ($K$) | 6 | 6 |
| Parallelism ($P$) | 0.5 | 0.5 |

Table 4: Parameters for Continuity Test Configurations.

The increment of a single user corresponds to a theoretical load increase of approximately 1.5% (1/64).

#### 4.3.2 Results Analysis

The simulation was executed with $R = 50$ independent runs. Since the same seeds were used for both configurations, we adopted the **Paired Observations** analysis method. For each run $i$, we computed the pointwise difference $Z_i = Y_i(N = 65) - X_i(N = 64)$ and performed a Paired T-Test to verify if the mean difference is significantly different from zero.

The results are summarized in Table 5.

| **Metric** | **Mean Diff ($\Delta$)** | **95% CI of Diff** | **P-Value** | **Verdict** |
|---|---|---|---|---|
| Response Time (s) | +0.0049 | $[0.0046, 0.0051]$ | $< 0.0001$ | **PASSED** |
| Waiting Time (s) | +0.0047 | $[0.0046, 0.0048]$ | $< 0.0001$ | **PASSED** |
| Server Utilization | +0.0114 | $[0.0113, 0.0115]$ | $< 0.0001$ | **PASSED** |
| Avg. Queue Length | +0.0325 | $[0.0317, 0.0332]$ | $< 0.0001$ | **PASSED** |

Table 5: Continuity Test Results (Paired Differences)

The analysis confirms the robustness of the simulator:

1. **Continuity:** The variations are extremely small (e.g., Response Time increased by only $\approx 5ms$) and consistent with the marginal load increase (Utilization increased by $\approx 1.14\%$).
2. **Monotonicity:** All mean differences are strictly positive. The P-Values ($< 0.0001$) indicate statistical significance, so the increase in metrics is due to the increased load and not to random noise.

#### 4.3.3 Visual Validation

Figure 6 shows the distribution of the paired differences for the key metrics.

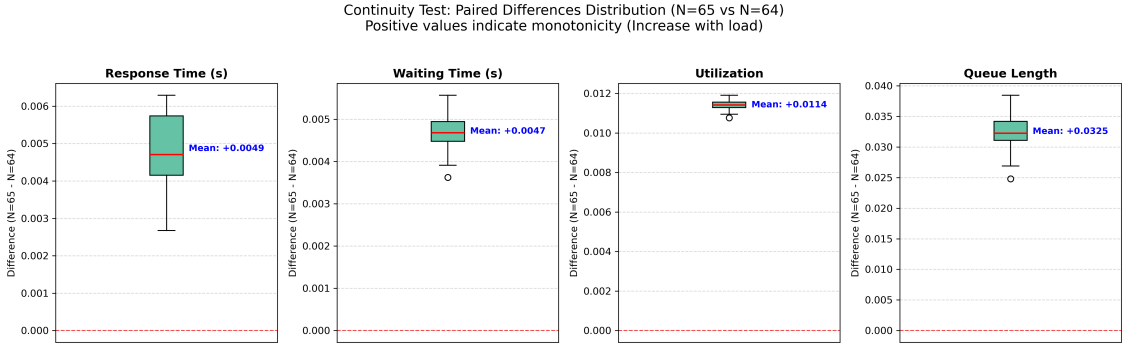



Figure 6: Distribution of Paired Differences ($N = 65; N = 64$). The strictly positive values confirm monotonicity. The narrow range confirms continuity.

The presence of outliers (dots) is expected in queuing systems due to the stochastic nature of arrival processes.

## 4.4 Validation through theoretical model

Results coming from theoretical model can be generally considered as an upper limit of our actual performance. However, the core difference evaluated within this M/M/c framework is how each model defines its service parameters and contributes to overall latency: the container-based model presents a single service event of duration $D$ plus fixed overhead $2I_{cont}$, while the FaaS model breaks the workload into $K$ sequential service events, each of duration $\frac{D}{K}$, accompanied by $K$ applications of the smaller overhead $2I_{FaaS}$. So we can compute $E[R]$ and $\rho$ for the two different cases and compare to the results from the simulation. To validate the correctness of the simulator's core queuing logic, we compared the simulation results with the analytical closed-form formulas for an $M/M/c$ system (Erlang-C). The scenario was configured with ideal conditions ($I_{cont} = 0, I_{FaaS} = 0$) and fixed request size ($R = 1$ GHz) to satisfy the $M/M/c$ assumptions.

### 4.4.1 Test Configuration

To ensure statistical significance while maintaining a realistic variance, we executed a group of $N = 50$ independent runs, each with a simulation time of 2 hours.

### 4.4.2 Analytical Reference

We computed the exact theoretical Mean Response Time $E[R]$ using the standard **Erlang-C** formula for the probability of waiting $P(wait > 0)$, combined with Little's Law. Based on the input parameters ($c = 40, \rho = 0.8, \mu = 0.2$), the theoretical expectation is:

$$E[R] = E[W] + \frac{1}{\mu} \approx 5.0757 \text{ s}$$

The calculation was performed using a validated Python script implementing the standard formulas for $M/M/c$ queues (which is available on our GitHub repository).

### 4.4.3 Results Analysis

Table 6 presents the comparison between the theoretical expectation and the empirical results that came from the simulation.

| Metric | Value |
|---|---|
| Theoretical Mean Response Time ($E[R]$) | 5.07574 s |
| Simulated Mean ($\bar{X}$) | 5.07991 s |
| Standard Deviation ($S$) | 0.03387 s |
| Sample Size ($runs$) | 50 |
| **95% Confidence Interval** | $[\mathbf{5.07052}, \mathbf{5.08930}]$ **s** |

Table 6: Theoretical Verification Results (N=50, Duration=$2h$).

#### 4.4.4 Discussion

The theoretical value (5.07574 s) falls strictly within the 95% confidence interval of the simulation results. With a relative error of 0.08%, we conclude that the simulator correctly implements the theoretical queuing dynamics and resource contention mechanisms modeled by Erlang-C.

### 4.5 Valgrind Test

The simulation executed successfully until reaching the configured simulation time limit at which point it terminated normally `finish()` and valgrind reports no memory leaks, as it is shown in Figure 7.

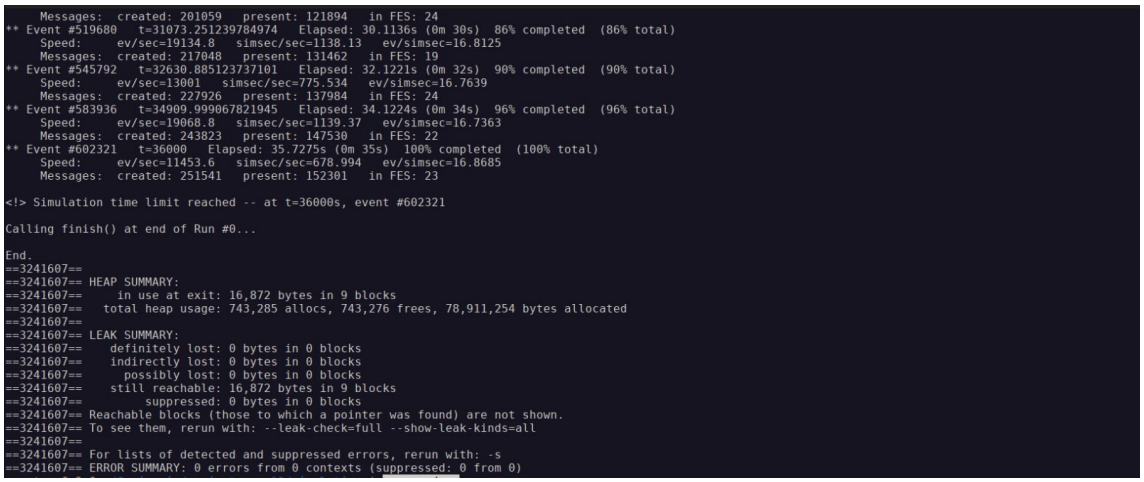



Figure 7: The output of Valgrind called on Omnet++ simulation executable.

## 5 Calibration

### 5.1 Cloud Computing Infrastructure Dimensioning

Since we need to put some numbers into our simulation, as a starting point we decided to find a set of physically plausible parameters, we have defined a large-scale **Private Data Center**, utilizing current hardware and scale specifications.

- **Data Center Scale (S):** The system is modeled as a cluster composed of **500 Physical Servers**. This value falls within the range of a "Small Data Center" (between 500 and 2,000 servers), according to informal IBM's guidelines. [1]
- **Reference Hardware:** The infrastructure is based on **Intel Xeon Gold 6138** processors, which represent enterprise-class hardware widely used in Cloud environments and for performance testing. [2]

#### 5.1.1 Server Specifications

Each server is configured with **two CPUs** (Dual-Socket architecture) to maximize compute density. The capacity is derived from the official **Intel Xeon Gold 6138** specifications [3]:

- **Number of cores per CPU: 20 Cores** per Socket.
- **Total cores ($C_s$):** 500 Servers × 2 CPUs × 20 Cores = 20,000 **equivalent virtual cores (vCPUs)**.
- **Nominal core capacity ($\mu$):** 2.00 GHz (We use this value, which represents the processor base frequency).

The **Total Aggregate Capacity (C)** of our system is then computed as follows:

$$C = 20{,}000 \text{ Cores} \times 2.00 \text{ GHz/Core} = 40{,}000 \text{ GHz}$$

This value is our realistic reference in order to dimension the parameters of our simulation.

#### 5.1.2 Physical-to-Virtual Resource Mapping

The system is modeled as a multi-server queue ($M/M/c$) where the service channels $c$ are mapped directly to the available computing cores (vCPUs) of the underlying physical infrastructure.

### 5.2 Factors Dimensioning

#### 5.2.1 Required Capacity ($R$)

**R** is defined as a **uniformly distributed** random variable, reflecting the varying demand of different cloud tasks. The selection of the range is based on the capacity of a single Virtual CPU (vCPU).

- **Capacity constraint:** The allocated capacity of a single vCPU is set at the processor's base frequency of 2.00 GHz. This represents the absolute maximum capacity a single workload can demand instantly.
- **Chosen range:** $R \sim U[0.5 \text{ GHz}, 1.5 \text{ GHz}]$.
- **Mean value ($E[R]$):** 1.0 GHz.
- **Methodological justification:** The range is set to be **below the vCPU maximum** ($R_{\max} = 1.5 \text{ GHz} < 2.0 \text{ GHz}$). This design choice:
  1. Ensures that no single workload instantaneously demands more than the capacity imposed by the Cloud system's allocation limit.
  2. Supports the **fluid capacity model**, ensuring that workloads occupy a fractional part of the total capacity pool, consistent with modern resource virtualization practices.

#### 5.2.2 Execution Duration ($D$)

$D$ is modeled as an **exponentially distributed** random variable for which we chose a mean value of $E[D] = 5.0$ seconds.

**Justification derived from the project specifications:** The project requires a comparative analysis between a Container-based model and a **Function as a Service (FaaS)** model .

- **FaaS Compatibility:** Real-world FaaS platforms are architecturally designed for quick, granular tasks. To make sure that the workload is valid for the FaaS model analysis, the duration **D** must fall within this short-term regime. While a container might run for much longer, we need to choose values that are compatible for both containers and FaaS platforms. Being the FaaS scenario more restrictive, than we decided to choose the execution time based on that particular case. This also allows us to make a fair comparison between the container-based model and the FaaS model, which wouldn't be compatible with long executions.

- **Realistic Reference:** Large-scale characterizations of workloads compatible with FaaS show that the vast majority of executions last between 1 and 10 seconds. [4]
- **Comparison Validity:** Setting $E[D] = 5.0s$ creates a realistic scenario where both long-term containers and short-term FaaS functions can be compared, highlighting the trade-offs in overhead ($I_{cont}$ vs $I_{FaaS}$) as requested by the project specifications.

#### 5.2.3 Dimensioning of Inter-Arrival Time ($T$)

**T** represents the time interval between consecutive workload submissions by a single user. We looked up some standard benchmarks that give a measure of **user think-time**. [5]

- **Distribution: Exponentially Distributed**, compliant with the Poisson arrival assumption.
- **Mean Value ($E[T]$):** 10.0 seconds.
- **Justification (TPC-W Standard):** This value is derived from the **TPC-W Benchmark** (Transaction Processing Performance Council), a standard for evaluating web-based transactional systems. Even if the standard is a bit old, we are using it to dimension the user think-time, as it is still valid when it comes to modeling **web user behavior**. TPC-W mandates an exponential user think time with a mean of 7.0 seconds. We decided to use a more conservative mean of 10.0 seconds as we are dealing with a generic cloud system that could be used by the employees of a company as they are doing their job, which typically involves slightly longer cognitive processing intervals than pure e-commerce transactions, while remaining in the same order of magnitude as the standard.

#### 5.2.4 Analytical Derivation of the User Range ($N$)

The size of the user population ($N$) is derived analytically as a dependent variable, determined by the system capacity ($C$) and the user behavior profile defined above.

**1. Average User Load Intensity** Based on the defined stochastic parameters ($R, D, T$):

$$\rho_{\text{user}} = \frac{E[R] \cdot E[D]}{E[T]} = \frac{1.0 \text{ GHz} \cdot 5.0 \text{ s}}{10.0 \text{ s}} = 0.5 \text{ GHz/user}$$

**2. System Saturation Point** Given the fixed physical capacity of $C = 40,000$ GHz, the number of users required to theoretically saturate the system ($\rho = 1$) is:

$$N_{\text{sat}} = \frac{40{,}000 \text{ GHz}}{0.5 \text{ GHz/user}} = 80{,}000 \text{ Users}$$

**3. Simulation Sweep Range** The simulation range is set to $N \in [5{,}000; 100{,}000]$. This sweep guarantees that the system is tested from a stable state ($\approx 6\%$ load) up to a congested state ($\approx 125\%$ load), allowing for the validation of the non-linear response time degradation.

#### 5.2.5 Derivation of the User Range ($N$)

The number of users $N$ is one of the two control parameters for the simulation. Its range is derived analytically to ensure the system is subjected to a complete load spectrum, from stability to saturation.

**1. Single User Load Intensity ($\rho_{\text{user}}$)** First, we calculate the average computing power demanded by a single active user. Based on the stochastic parameters defined previously:

$$\rho_{\text{user}} = \frac{E[R] \cdot E[D]}{E[T]} = \frac{1.0 \text{ GHz} \cdot 5.0 \text{ s}}{10.0 \text{ s}} = 0.5 \text{ GHz/user}$$

This indicates that, on average, each user continuously occupies 0.5 GHz of the system capacity.

**2. Theoretical Saturation Threshold ($\mathbf{N_{sat}}$)** Using the Total Aggregate Capacity of the reference data center ($C = 40,000$ GHz), we determine the exact number of users required to reach 100% utilization ($\rho = 1$):

$$N_{sat} = \frac{C}{\rho_{\text{user}}} = \frac{\text{40,000 GHz}}{\text{0.5 GHz/user}} = 80000 \text{ Users}$$

**3. Simulation Sweep Definition** To visualize the non-linear degradation of response times (queue build-up), the simulation must extend well beyond the saturation point. The range is defined as follows:

- **Starting point:** $N = 5{,}000$ (Load $\rho \approx 0.06$). Represents the baseline performance with negligible queuing.
- **Critical region:** $N \approx 64{,}000$ (Load $\rho = 0.80$). The point where exponential delay growth typically begins.
- **End point:** $N = 100{,}000$ (Load $\rho = 1.25$). Represents an overloaded system to demonstrate failure behavior and infinite queue growth.

The simulation will therefore perform a sweep of $N$ in the interval $[5{,}000; 100{,}000]$.

### 5.2.6 Dimensioning of Function Granularity ($K$)

The parameter $K$ defines the number of sequential functions into which a single workload is decomposed in the FaaS model. The simulation evaluates system performance across a discrete set of $K$ values to quantify the trade-off between granularity and cumulative overhead.

- **Sweep Values:** $K \in \{2, 3, 4, 5, 6, 7, 8\}$.
- **Typical Scenarios for different Values:**
  1. **K = 1 (Baseline):** Corresponds to the **container-based model** (Monolithic execution). This is the reference point where overhead is minimized ($2 \times I_{cont}$).
  2. **K = 3 (Typical FaaS):** Represents the median composition of real-world FaaS applications, as identified in large-scale empirical studies of FaaS platforms. [6]
  3. **K = 5 (Complex workflow):** Represents a moderately complex pipeline.
  4. **K = 8 (High fragmentation):** An extreme scenario used to stress-test the system, highlighting the impact of cumulative initialization overhead ($2 \times K \times I_{FaaS}$) on the total execution time.

### 5.2.7 Dimensioning of Instantiation and Termination Overheads ($I_{cont}$ and $I_{FaaS}$)

The instantiation overheads represent the setup and destruction times required by the virtualization technology. To ensure a meaningful comparison where we don't have one specific model that dominates across the entire range of $K$, we calibrated these parameters to mantain a real-world scenario but also to ensure a fair comparison between the two modes. As described in this project's specifications, the instantiation time is the same as the termination time, so in the following paragraphs we are simply calibrating the **Instantiation Time**

**1. FaaS Instantiation Time ($I_{FaaS}$)** Agache et al. [7] report AWS Firecracker MicroVM boot times of $\approx 125$ ms:

$$I_{FaaS} = 0.125 \text{ seconds}$$

**2. Container Instantiation Time ($I_{cont}$)** For the container model, benchmarks in [8] show Docker startup times starting from 0.6s - 0.9s for highly optimized, lightweight containers. To represent a highly optimized container scenario and maintain consistency with the short workload duration ($E[D] = 5.0$s), we selected:

$$I_{cont} = 0.800 \text{ seconds}$$

| Parameter | Value | Rationale |
|---|---|---|
| $I_{FaaS}$ | 0.125 s | MicroVM Boot [7] |
| $I_{cont}$ | 0.800 s | Optimized Docker Warm-Start [8] |

Table 7: Instantiation Overheads Calibration

## 5.3 Scaled Model for Efficient Simulation

While the dimensioning presented in the previous sections establishes a realistic baseline linked to physical hardware, simulating a system with 20,000 active service channels and 100,000 users imposes significant computational overhead. In order to be able to perform multiple simulation runs within the project constraints, we introduce a **Scaled Simulation Model**.

### 5.3.1 Scaling Methodology

Our scaling strategy applies a reduction factor ($\alpha$) to the extensive properties of the system (Total Capacity, Number of Servers, Population Size) while preserving the intensive properties (Workload Statistics, Unit Core Speed). This ensures that the **Traffic Intensity** ($\rho$) and the resulting performance curves (Response Time vs. Utilization) remain statistically consistent with the reference scenario.

- **Scaling Factor ($\alpha$):** We select $\alpha = 1000$.
- **Invariant Parameters:** The stochastic behavior of the users and workloads remains unchanged ($R, D, T, \mu_{\text{unit}}$). This preserves the fundamental interaction dynamics between tasks and resources.

### 5.3.2 Scaled Parameters

Applying the factor $\alpha = 1000$, the simulation parameters are transformed as follows:

1. **Scaled Total Capacity ($\mathbf{C'}$):**
$$C' = \frac{C}{\alpha} = \frac{\text{40,000 GHz}}{1000} = 40 \text{ GHz}$$
2. **Scaled Service Channels ($\mathbf{C'_s}$):**
$$C'_s = \frac{C_s}{\alpha} = \frac{\text{20,000 vCPUs}}{1000} = 20 \text{ vCPUs}$$
3. **Scaled User Range ($\mathbf{N'}$):** The user population is scaled proportionally to maintain the same Load Factor ($\rho_{\text{user}} = 0.5$):
$$N'_{\text{range}} = \left[\frac{5{,}000}{1000}; \frac{100{,}000}{1000}\right] = [5; 100] \text{ Users}$$
4. **Verification of Saturation Point:** The new saturation point ($\rho = 1$) occurs at:
$$N'_{\text{sat}} = \frac{C'}{\rho_{\text{user}}} = \frac{40 \text{ GHz}}{0.5 \text{ GHz/user}} = 80 \text{ Users}$$
This confirms that the scaled system retains the exact same utilization profile as the reference system (80 is to 100 as 80,000 is to 100,000).

### 5.3.3 Refinement of the Simulation Range

Although the theoretical scaling derived above suggests a full sweep range of $N' \in [5, 100]$, we decided to narrow this range in order to optimize our simulations, since the very extreme values are not going to be very informative. This also improves the readability of the performance curves in the critical region, so we narrowed the simulation interval to:

$$\mathbf{N_{sim}} = [20, 90] \text{ Users}$$

| **Parameter** | **Reference Scenario** | **Scaled Simulation** ($\alpha = 1000$) |
|---|---|---|
| Total Capacity ($C$) | 40,000 GHz | **40** GHz |
| Total vCPUs ($k$) | $20,000$ | **20** |
| Saturation Users ($N_{sat}$) | $80,000$ | **80** |
| System Users ($N$) | $[5k; 100k]$ | $[\mathbf{20}; \mathbf{90}]$ |

Table 8: Comparison: Reference vs. Scaled Scenario

## 5.4 Warm-up time

The evolution of performance indices during the initial simulation phase was studied to calibrate the warm-up duration. The results indicate that **response time** serves as the superior metric for this purpose; unlike waiting time, which can be null, response time consistently provides measurable values. For all the experiments conducted, it was decided to set the simulation time to 10,000$s$ allowing to collect enough data for retreiving meaningful conclusions. After setting all the parameters, and in particular $p = 0.5$ as the probability to generate the workload as FaaS in order to create more or less balanced results, we found that a general warm-up time is 500 seconds. We initially analyzed the response time using a sliding window average (window size = 200 samples), as shown in Figure 8. However, as seen in the graph, this method still showed significant oscillations. Consequently, we adopted a second approach by plotting the cumulative time average of the response time, as shown in Figure 9. Based on Figure 9, we observed that the average response time stabilizes and flattens out after approximately 500 seconds. Therefore, we defined $T_{warmup} = 500\ s$. Figure 10 shows the new plot without the warm-up period.

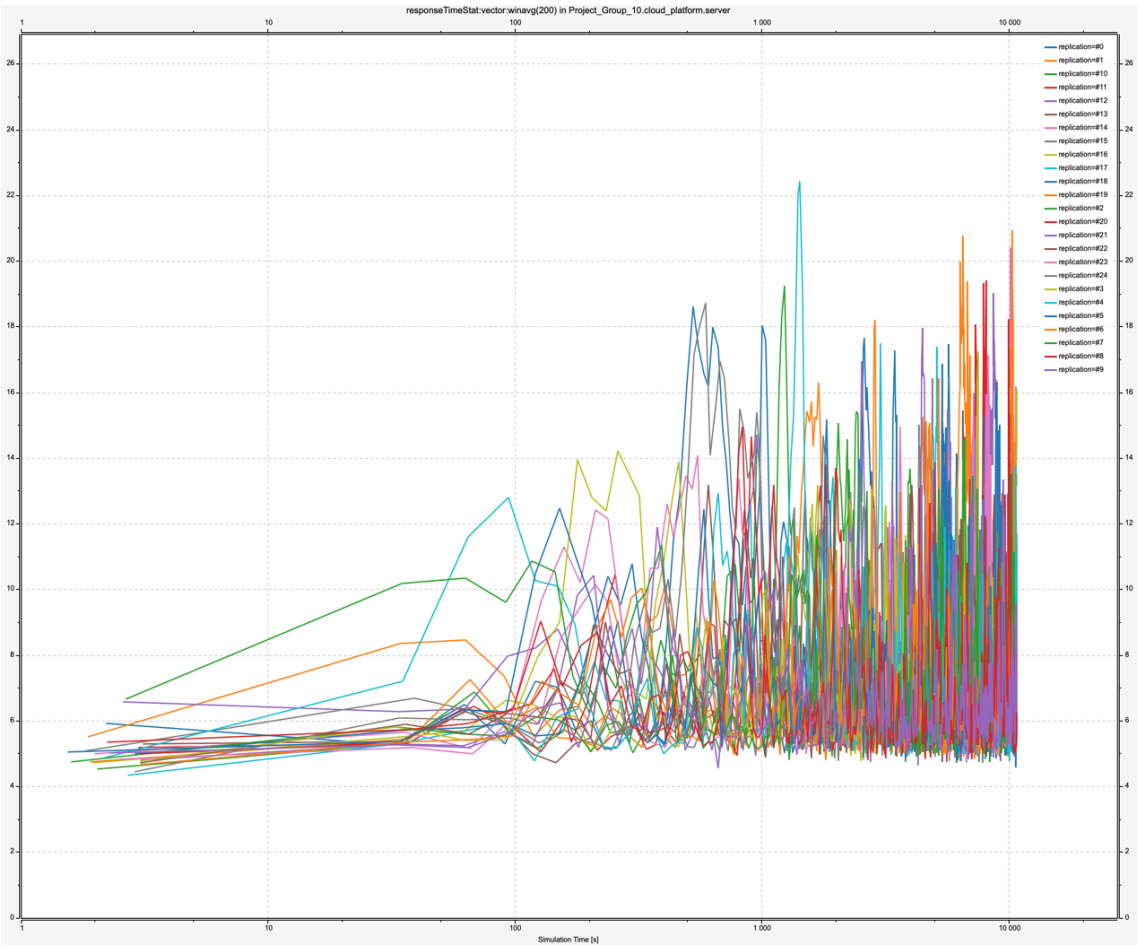



Figure 8: Response-time plotted a sliding window average of 200 elements.

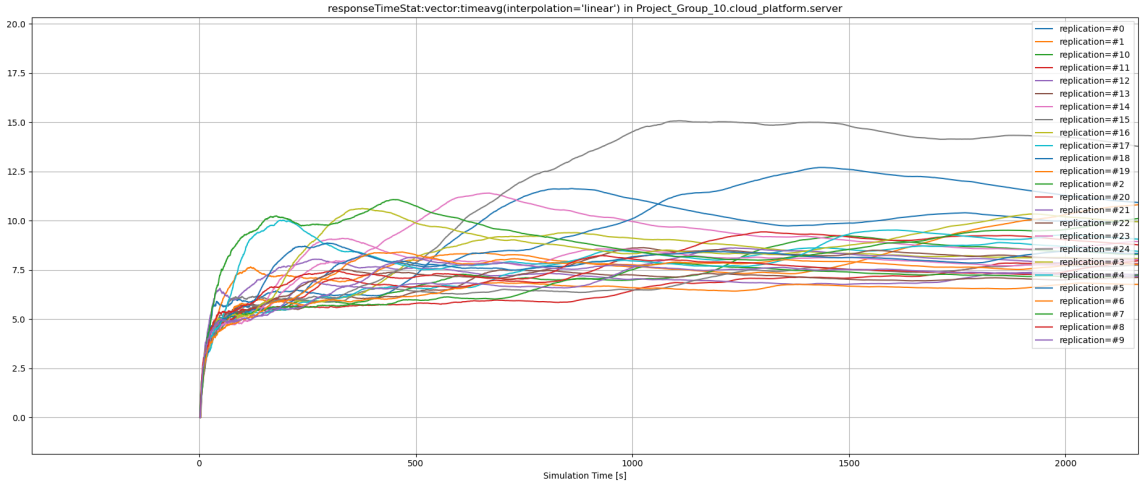



Figure 9: Response-time plotted a time average.

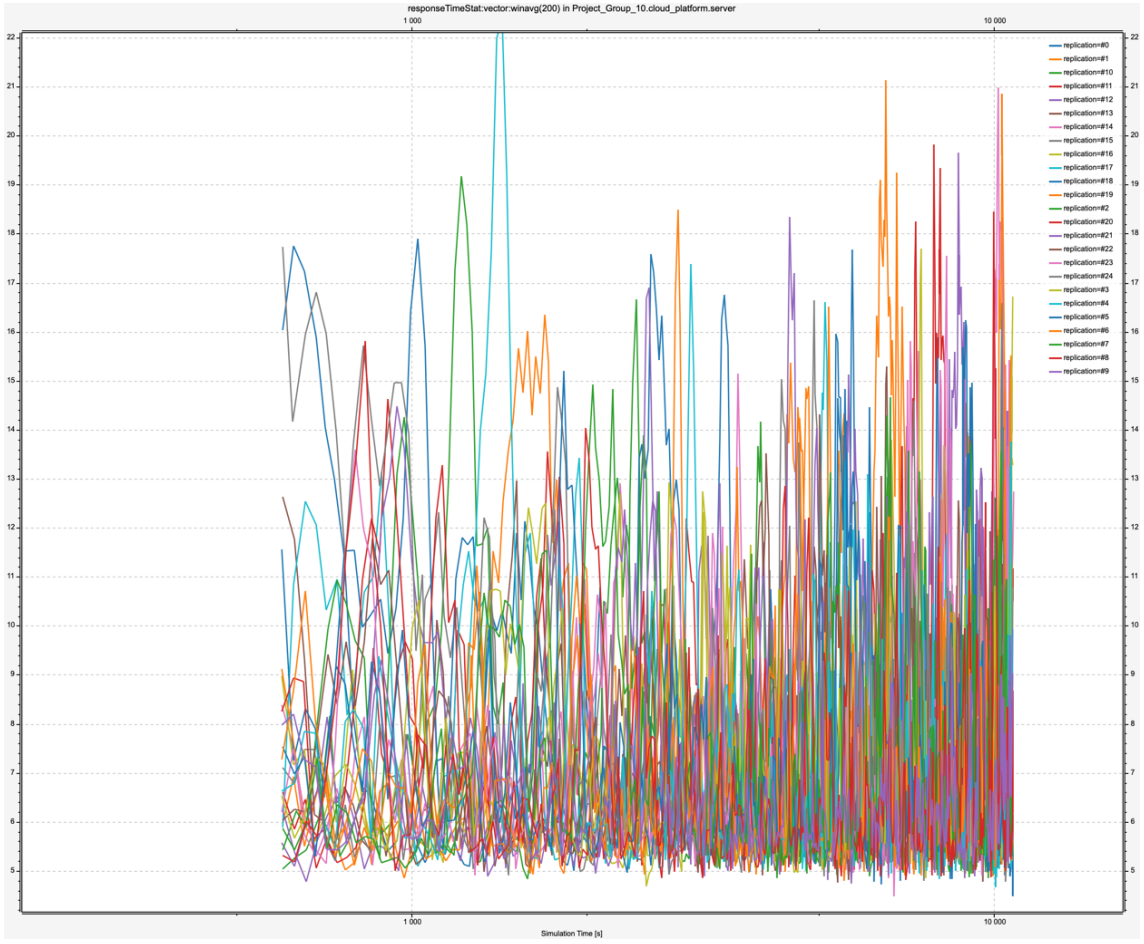
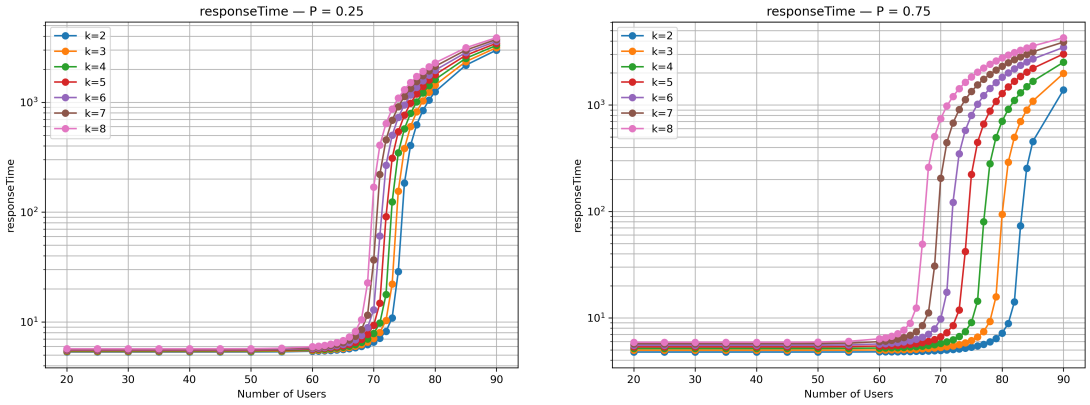



Figure 10: Response time plot without the warm-up period.

# 6 Simulation

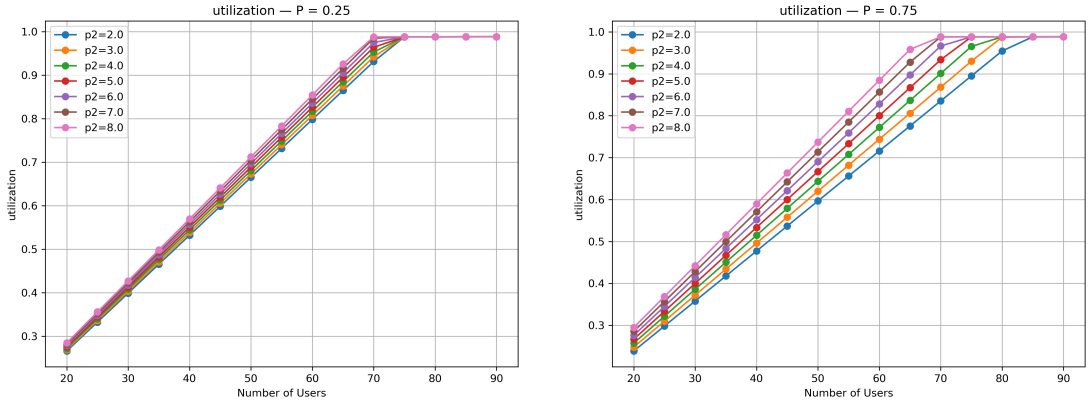
For the simulation experiments, we used different values of $p$ to check the impact of the number of container-based requests and FaaS-based requests, comparing the utilization and the response

time to check which model is convenient to used depending on the values of $N$ and $K$. Figure 11 shows the comparison between the response time with $p = 0.25$, meaning that the most requests are container-based, and with $p = 0.75$. As shown in subfigure 11b, when $K$ grows, the saturation point is reached with **few users** . This occurs because in the FaaS model, each additional function introduces a initialization overhead $I_{FaaS}$ that consumes faster the system. Figure 12, instead, shows system utilization as a function of number of users. Utilization seems to grow linearly as users increase, however, comparing subfigure 12a and subfigure 12b, it's evident that with a higher FaaS presence, the curves are steeper. Similarly, greater value of $K$ lead to faster saturation.




(a) Response time plot with $p = 0.25$ (b) Response time plot with $p = 0.75$

Figure 11: Comparison of the impact of FaaS requests and container-based requests in response time




(a) Utilization plot with $p = 0.25$ (b) Utilization plot with $p = 0.75$

Figure 12: Comparison of the impact of FaaS requests and container-based requests in utilization

# 7 Conclusions

In conclusion, our simulations evidence that the choice between these models is fundamentally governed by the trade-off between instantiation overhead and function granularity. The simulation results clearly demonstrated that the FaaS model is highly sensitive to workload fragmentation. However, it is essential to highlight a significant limitation in our comparison: the independence of

overheads from container "weight". Our model treats $I_{cont}$ and $I_{FaaS}$ as fixed constants. In a real-world scenario, these values are not static but are strictly dependent on the size and complexity of the software. Future iterations of this model should introduce a variable weight factor $W$ to dynamically scale the overheads, providing a more granular evidence of where the efficiency gap between serverless and containers truly lies.

# References


[1] IBM, "What is a hyperscale data center?" https://www.ibm.com/it-it/think/topics/hyperscale-data-center, 2025.

[2] M. Tofigh, X. Ding, and E. Guo, "Optimizing CPU Cache Utilization in Cloud VMs with Accurate Cache Abstraction," *arXiv preprint arXiv:2511.09956*, 2025, submitted to arXiv, November 2025.

[3] Intel, "Intel xeon gold 6138 processor - specifications," https://www.intel.com/content/www/us/en/products/sku/120476/intel-xeon-gold-6138-processor-27-5m-cache-2-00-ghz/specifications.html, 2025.

[4] M. Shahrad, R. Fonseca, Í. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider," 2020. [Online]. Available: https://www.usenix.org/conference/atc20/presentation/shahrad

[5] *TPC Benchmark W (Web Commerce) Specification, Version 1.8*, Transaction Processing Performance Council (TPC), February 2002, section 5.3.1.1: User Think Time. [Online]. Available: http://www.tpc.org/tpcw/

[6] S. Eismann, J. Scheuner, E. van Eyk, M. Schwinger, J. Grohmann, N. Herbst, C. L. Abad, and A. Iosup, "Serverless Applications: Why, When, and How?" *IEEE Software*, vol. 38, no. 1, 2020, analysis of serverless application structures finding a median function composition size of 3-4.

[7] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight Virtualization for Serverless Applications," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020, santa Clara, CA, February 2020.

[8] H. Dinh-Tuan, "Unikernels vs. Containers: A Runtime-Level Performance Comparison for Resource-Constrained Edge Workloads," *arXiv preprint arXiv:2509.07891*, 2025, submitted to arXiv, September 2025.